

2ND Session: Understanding Laravel Structure, Routing, and Views

OVERVIEW:

Now that Laravel is installed and running, students will begin to understand:

1. What Laravel generated?
2. How Laravel handles requests?
3. How routing works?
4. How views are displayed?
5. How MVC works in real files?

In this module, we will focus mainly on understanding, not memorization.

Learning Objectives:

At the end of this session, students should be able to:

1. Identify important Laravel folders and files
2. Explain the basic Laravel request flow
3. Create simple routes
4. Display text using routes
5. Create and return Blade views
6. Understand the connection between Route and View

2.1 UNDERSTANDING THE LARAVEL PROJECT STRUCTURE

2.1.1. What did Laravel Create?

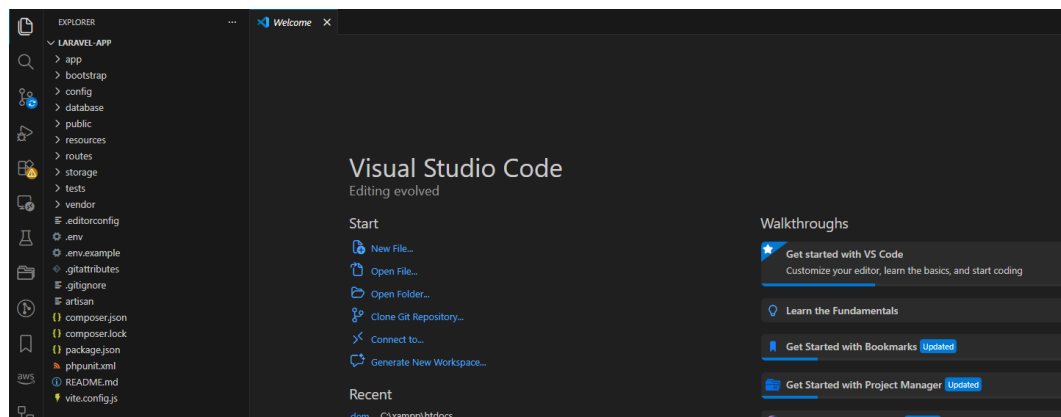
When we successfully installed the Laravel and created a Laravel New “Laravel-app” folder inside our first Laravel app folder, and type **code .** and hit enter, this opens your VS Code editor automatically with your Laravel-app project. In this process, Laravel generated a complete web application structure for us. Professional developers use this structure in their project.

2.1.2. What are these folders?

In the image below, shows the Laravel-app project created. As noticed, it has several folders, files with different file format such as .json, .lock, .xml, .js, etc and different function.

Think of the Laravel-app project as an institution (MPSU). Each folder represents a school. And every school has specific course, programs, specific responsibility.

Don't panic. You don't need to understand everything today. Let's take this step by step. For this module or activity, we will mainly focus on the important folders such as app, routes, resources, public, and so on.



a. **app/**

This folder is known as the brain of your application. This is where your system's intelligence lives. Inside this folder, you will mostly work with

1. **controllers**

This controller serves as THE DECISION MAKERS. It is a file that handles user requests, processes logic, and returns a response (usually a view). For example, when a user visits your site. Let's say /profile. What controller do is it will get user data, send it to a view, and display profile page.

In simple or single page app, in Laravel, you don't need to create controller, simply create a route. Controller becomes necessary when:

1. You want cleaner structure
2. You have many pages
3. You are processing forms
4. You are working with database
5. You are building complex applications.

*** Imagine you have 30 routes. Its messy to many. Then that's when controller becomes necessary.

How to create controller?

Php artisan make:controller ProfileController

Inside this controller,

```
class ProfileController extends Controller
{
    public function show()
    {
        $name = "Juan Dela Cruz";
        return view('profile', ['name' => $name]);
    }
}
```

Then route:

```
Route::get('/profile', [ProfileController::class, 'show']);
```

2. **models**

this is what they call the Data experts. It represents a database table, a data structure, a real-world entity. For example, if your system is about students, then Student is a model. If its about product, then Product is a model.

Let's say you created a table in your database called "students".

You create a model using:

```
php artisan make:model Student
```

Now, in this case, Laravel understands Student model <-> student table. Later on, you will use this for saving data, retrieving, updating, and deleting data (CRUD). In short, Model is responsible in communicating to the database.

3. Application Logic

This simply means any logic that makes your system behave intelligently. For example, calculating grades, validating forms, computing totals, applying discounts, checking if user is allowed to login, etc.

This kind of logic lives in Controllers, models, application logic or other classes inside the folder app/.

b. routes/

This folder serves as the traffic controller. This file decides what happens when someone types a URL. For example, let's take your e-commerce website activity in your web systems and technologies subject. Your website consists of different pages like home, about, contact us, cart, etc. Here in Laravel, for you to view the created file or web page let's say about page, just go to routes and add this page. How?

```
Route::get('/about', function () {  
    Return view ('about')  
});
```

This means if someone goes to /about, it will show the about page.

In PHP, you simply use <a href> to connect this. But for Laravel, you need to include this in Route for it to be accessible to be viewed or presented in the resources/views.

c. resources/views

This is the presentation layer. This is where the design lives. The files end with .blade.php. We call these Blade templating. This is easy since you already have an idea about php. For the pure php, we just simply use .php but for this laravel framework, we need to use the proper blade templating for us to access the page.

VIEW – what is this view all about? This view displays HTML, shows data, or shows what users should see. Take note, it does not handle database or make decisions. It only represents information. It's the front-end, what the users can see. In addition, view displays only data given by the controller.

In short, serves as folders of our front-end (css, js, views (contains html and php file)). So if you want to add custom js for your system or web app, just go to resources/views and save your js file. Same is thru if you want o create your own css,

d. Database

This is where data structure is defined. There are important subfolders here when you visit the database/. It has migrations/, seeders/, and factories. Later on, we will talk more about migrations when we proceed with our actual project creation.

e. Public

This is the only folder accessible to the browser. It contains index.php which serves as the entry point. CSS, images, and JS. TAKE NOTE THAT THE BROWSER NEVER DIRECTLY ENTERS YOUR APP FOLDER, IT ENTERS THROUGH PUBLIC.

f. .env

Accordingly, this file contains environment settings like: database name, database username, App URL, and App key. Later on, if you will create your project, this will be included but I suggest to never upload this to GitHub.

Model handles data,
View handles display,
Controller handles decision-making.

Basic routing + view – for concept or single page site. Beginner approach or method.

QUESTION: Where exactly do we write routing? Where do we write view?

To sum it up. Let's say we are to create a Learning Management System. First is to create a View (presentation layer).

1. Go to resources/views – create homepage.blade.php.
2. Inside your homepage.blade.php file, provide the html boilerplate for your design later on.

Note: if we will access this one. php artisan serve. Copy the provided URL

<http://127.0.0.1:8000/homepage.blade.php>

Take note that this will not work yet. Why? Because Laravel controls access through routes. There is something missing. What is it? The route.

3. Create a route that returns the view. Go to routes/web.php – this is where you define URLs. So whenever you want to define a new page URL, just go to routes/web.php
4. Copy and paste the default route. Just rename it to the /homepage and return view to homepage.

This:

```
Route::get('/homepage', function () {  
    return view('homepage');  
});
```

Or this:

```
Route::get('sample', function () {  
    return('sample');  
});
```

5. Test in Browser. <http://127.0.0.1:8000/homepage>

Routing + controller + view – for complex project. Professional Way.

QUESTION: Where exactly do we write routing? Where do we write controller? Where do we write view?

1. Go to VS code and click terminal > new terminal. Make sure you are inside your Laravel-app project folder in your terminal.

Note: you do not manually create it inside random folders. Laravel already has a proper location for controllers located in your app/Http/Controllers/ - this is the official folder for controllers.

2. Create the controller. Type: php artisan make:controller PageController then enter.
3. Check created controller in your controller folder.
4. Add a method inside the controller.

```
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;

class PageController extends Controller
{
    public function homepage()
    {
        return view('homepage');
    }
}
```

5. Create the view. Go to resources/views. Create new file name it homepage.blade.php
6. Connect route to Controller. Go to routes/web.php.
7. At the top add:
use App\Http\Controllers\PageController;
8. Then add your route:
Route::get('/homepage', [PageController::class, 'homepage']);
9. Save.
10. Run the server. In terminal write php artisan serve then open the provided URL.

SO THIS IS MVC IN ACTION: Route > Controller > View > browser

EXERCISE: SIMPLE LEARNING MANAGEMENT SYSTEM

1. CREATE A SEPARATE FOLDER FOR YOUR LMS. Resources/views/lms. Inside lms folder create your blade.php files:
 - a. Dashboard.blade.php
 - b. Courses.blade.php
 - c. Course-details.blade.php
 - d. Students.blade.php
 - e. Enroll.blade.php
 - f. enrollSubmit()
2. Use One controller. Create controller. php artisan make:controller LMSSController. Create a methods like:
 - a. dashboard()
 - b. courses()
 - c. courseDetails(\$id)
 - d. students()
 - e. enrollForm()
 - f. enrollSubmit()
3. Group routes with prefix. In routes/web.php.
use App\Http\Controllers\LMSSController;

```
Route::prefix('lms')->group(function () {
    Route::get('/', [LMSSController::class, 'dashboard']);
    Route::get('/courses', [LMSSController::class, 'courses']);
    Route::get('/courses/{id}', [LMSSController::class, 'courseDetails']);
    Route::get('/students', [LMSSController::class, 'students']);
    Route::get('/enroll', [LMSSController::class, 'enrollForm']);
    Route::post('/enroll', [LMSSController::class, 'enrollSubmit']);
});
```

4. Return view. Go to app/Http/Controllers/LMSSController.php, then add return view. Just add other method.

```
<?php
```

```
namespace App\Http\Controllers;
```

```
use Illuminate\Http\Request;
```

```
class LMSSController extends Controller
```

```
{
    public function dashboard()
    {
        return view('lms.dashboard');
    }
}
```

```
public function courses()
{
    return view('lms.courses');
}
```

```
public function courseDetails($id)
```

```

{
    return view('lms.course-details', [
        'id' => $id
    ]);
}

public function students()
{
    return view('lms.students');
}
public function enrollForm()
{
    return view('lms.enroll');
}
public function enrollSubmit(Request $request)
{
    return "Enrollment submitted successfully!";
}
}

```

Previously in our basic routing exercise, we put `return view()` inside `routes/web.php`. this time, we will not put. Why? Because routes are pointing to controller methods.

routes = map URL. Only decide who handles the request.

Controller = return view. Decide what to return

View = display content. Decide how it looks

So, in this exercise, our routes file only tells Laravel which controller method to run. Then controller method is responsible for returning the view.